Ushtrime
Programimi i orientur drejt objekteve (OOP)

Alion Emini

kurseinformatike.com

Përmbajtja

1. Pyetje me alternativa
2. Pyetje True/False
3. Pyetje të shkurtra
4. Algoritme
5. Ushtrime në kod

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Multiple Choice #1

Pyetja. Cila është praktika e programimit që është e fokusuar në krijimin e funksioneve të ndara nga të dhënat që ato përdorin?

a) modular
b) procedurale
c) funksionale
d) orientuar nga objektet

Multiple Choice #1 - Zgjidhja

Zgjidhja: c) funksionale

Paradigma funksionale e programimit është e fokusuar në përdorimin e funksioneve si ndërtuesit kryesorë, të ndara nga të dhënat.

Multiple Choice #2

Pyetja. Cila është praktika e programimit që është e fokusuar në krijimin e objekteve?

a) objekt-centër
b) objektiv
c) procedurale
d) orientuar nga objektet

Multiple Choice #2 - Zgjidhja

Zgjidhja: d) orientuar nga objektet

Paradigma orientuar nga objektet është e fokusuar në krijimin e objekteve, të cilat janë instanca të klasave.

Multiple Choice #3

Pyetja. Çfarë është një komponent i një klase që i referohet të dhënave?

a) metodë
b) instancë
c) atribut i të dhënave
d) modul

Multiple Choice #3 - Zgjidhja

Zgjidhja: c) atribut i të dhënave

Atributet e të dhënave mbajnë vlera në një objekt, ndërsa metodat janë funksione që lidhen me një objekt.

Multiple Choice #4

Pyetja. Çfarë është një objekt?

a) model
b) formë
c) variabël
d) instancë

Multiple Choice #4 - Zgjidhja

Zgjidhja: d) instancë

Një objekt është një instancë e një klase, që përmban atributet dhe metodat e saj.

Multiple Choice #5

Pyetja. Si mund të fshehni një atribut të një klase nga kodi jashtë klasës?

a) duke shmangur përdorimin e parametrave self për të krijuar atributin
b) duke filluar emrin e atributit me dy nënvizime
c) duke filluar emrin e atributit me ___
d) duke filluar emrin e atributit me simbolin @

Multiple Choice #5 - Zgjidhja

Zgjidhja: b) duke filluar emrin e atributit me dy nënvizime

Kjo është mënyra e Python për të shënuar një atribut si privat dhe të paaksesueshëm jashtë klasës.

Multiple Choice #6

Pyetja. Cila metodë merr vlerën e një atributi të të dhënave por nuk e ndryshon atë?

a) marrës
b) konstruktor
c) modifikues
d) aksesor

Multiple Choice #6 - Zgjidhja

Zgjidhja: d) aksesor

Metodat aksesor përdoren për të marrë vlerat e atributeve pa i modifikuar ato.

Multiple Choice #7

Pyetja. Cila metodë ruan një vlerë në një atribut të të dhënave ose e ndryshon atë në një mënyrë tjetër?

a) modifikues
b) konstruktor
c) modifikues
d) aksesor

Multiple Choice #7 - Zgjidhja

Zgjidhja: c) modifikues

Metodat modifikues përdoren për të modifikuar vlerën e atributeve të një objekti.

Multiple Choice #8

Pyetja. Cila metodë thirret automatikisht kur krijohet një objekt?

a) `__init__`

b) `init`

c) `__str__`

d) `__object__`

Multiple Choice #8 - Zgjidhja

Zgjidhja: a) `__init__`

Metoda `__init__` është metoda e konstruktorit në Python, që thirret kur krijohet një objekt.

Multiple Choice #9

Pyetja. Nëse një klasë ka një metodë të quajtur `__str__`, cilat janë disa nga mënyrat për ta thirrur atë metodë?

a) e thirrni si çdo metodë tjetër: `object.__str__()`

b) duke kaluar një instancë të klasës në funksionin e brendshëm str

c) metoda thirret automatikisht kur krijohet objekti

d) duke kaluar një instancë të klasës në funksionin e brendshëm state

Multiple Choice #9 - Zgjidhja

Zgjidhja: b) duke kaluar një instancë të klasës në funksionin e brendshëm `__str__`

Metoda `__str__` thirret automatikisht kur një objekt kalon në funksionin `__str__`.

Multiple Choice #10

Pyetja. Cila është gjuha e modelimit të unifikuar (UML)?

a) Gjuhën e Modelimit të Unifikuar
b) diagramet e rrjedhës
c) pseudokodi
d) Sistemi i Hierarkisë së Objektit

Multiple Choice #10 - Zgjidhja

Zgjidhja: a) Gjuhën e Modelimit të Unifikuar
UML ofron diagrame të standardizuara për vizualizimin dhe modelimin e sistemeve orientuar nga objektet.

Multiple Choice #11

Pyetja. Cila është mënyra për të identifikuar klasat në një përshkrim të problemit?

a) foljet
b) mbiemrat
c) përemrat
d) emrat

Multiple Choice #11 - Zgjidhja

Zgjidhja: d) emrat

Në dizajnin orientuar nga objektet, klasat zakonisht identifikohen nga emrat në përshkrimin e problemit, që zakonisht përfaqësojnë objekte ose entitete.

Multiple Choice #12

Pyetja. Cila është mënyra për të identifikuar atributet dhe metodat e një klase?

a) përgjegjësitë
b) emri
c) sinonimet
d) emrat

Multiple Choice #12 - Zgjidhja

Zgjidhja: a) përgjegjësitë

Përgjegjësitë e një klase ndihmojnë në përcaktimin e atributeve dhe metodave që ajo duhet të ketë.

Multiple Choice #13

Pyetja. Në një marrëdhënie trashëgimie, cila është klasa e përgjithshme?

a) nënklasa
b) klasa e bazës
c) klasa robote
d) klasa fëmijë

Multiple Choice #13 - Zgjidhja

Zgjidhja: b) klasa e bazës

Klasa e bazës është klasa më e përgjithshme, ndërsa nënklasa është më e specializuar.

Multiple Choice #14

Pyetja. Në një marrëdhënie trashëgimie, cila është klasa më e specializuar?

a) klasa e bazës
b) klasa e majtë
c) nënklasa
d) klasa prind

Multiple Choice #14 - Zgjidhja

Zgjidhja: c) nënklasa

Nënklasa është një klasë më e specializuar që trashëgon atribute dhe metoda nga klasa e bazës.

Multiple Choice #15

Pyetja. Në një program që përdor dy klasa: `Avion` dhe `JumboJet`, cila nga ato është më e mundshme të jetë nënklasa?

a) `Avioni`
b) `JumboJet`
c) Të dyja
d) Asnjëra

Multiple Choice #15 - Zgjidhja

Zgjidhja: b) `JumboJet`

`JumboJet` mund të jetë një nënklasë që zgjerohet nga `Avioni`, duke shtuar veçori të tjera specifike.

Multiple Choice #16

Pyetja. Cila është një karakteristikë e programimit orientuar nga objektet që lejon thirrjen e versionit të duhur të një metode të kaluara në mënyrë të mbivendosur kur një nënklasë përdor një metodë të nënklasuar?

a) polimorfizëm
b) trashëgimi
c) generalizim
d) specializim

Multiple Choice #16 - Zgjidhja

Zgjidhja: a) polimorfizëm

Polimorfizmi mundëson thirrjen e metodës së saktë nga një nënklasë, pavarësisht nga referenca e klasës bazë.

Multiple Choice #17

Pyetja. Si mund të kontrolloni nëse një objekt është një instancë e një klase?

a) duke përdorur operatorin `in`

b) duke përdorur funksionin `is_object_of`

c) duke përdorur funksionin `isinstance`

d) duke përdorur mesazhet e gabimeve që shfaqen kur një program përplaset

Multiple Choice #17 - Zgjidhja

Zgjidhja: c) duke përdorur funksionin `isinstance`

Funksioni `isinstance` përdoret për të kontrolluar nëse një objekt është instancë e një klase të caktuar.

Multiple Choice #18

Pyetja. Cila është veçoria e rëndësishme që e bën një klasë të quhet "klasë bazë" në një marrëdhënie trashëgimie?

a) Atributet janë të fshehura për nënklasat
b) Nënklasat nuk mund të trashëgojnë metodat
c) Ofron funksionalitete të përgjithshme për nënklasat
d) E ka metoden __init__

Multiple Choice #18 - Zgjidhja

Zgjidhja: c) Ofron funksionalitete të përgjithshme për nënklasat

Klasa bazë ofron funksionalitete që mund të përdoren ose të mbivendosen nga nënklasat.

Multiple Choice #19

Pyetja. Çfarë do të thotë inkapsulimi në programimin orientuar nga objektet?

a) Mbështetje për kodin modular

b) Fshehja e të dhënave dhe ekspozimi vetëm i metodave të kontrolluara

c) Përdorimi i funksioneve si ndërfaqe për ndërveprimin me objektet

d) Lejon thirrjen e metodave pa pasur nevojë për instancim të objektit

Multiple Choice #19 - Zgjidhja

Zgjidhja: b) Fshehja e të dhënave dhe ekspozimi vetëm i metodave të kontrolluara

Inkapsulimi është një nga parimet kryesore të OOP dhe e siguron mbrojtjen e të dhënave duke fshehur ato dhe duke ekspozuar vetëm metodat që mund të manipulojnë ato.

Multiple Choice #20

Pyetja. Cili është roli i metodës __init__ në një klasë Python?

a) Të krijojë një objekt të ri të klasës
b) Të inicializojë atributet e klasës për secilin objekt të ri
c) Të bëjë polimorfizëm në nivel të metodave
d) Të krijojë lidhje me një bazë të dhënash për objektet

Multiple Choice #20 - Zgjidhja

Zgjidhja: b) Të inicializojë atributet e klasës për secilin objekt të ri

Metoda __init__ thirret kur një objekt i ri krijohet dhe përdoret për të inicializuar atributet e tij.

Multiple Choice #21

Pyetja. Cila është diferenca midis një variabli instancë dhe një variabli klasë në Python?

a) Variablat e klasës janë të disponueshëm për të gjitha instancat e klasës, ndërsa variablat e instancës janë specifike për çdo objekt

b) Variablat e klasës janë specifike për çdo objekt, ndërsa variablat e instancës janë të disponueshme globalisht

c) Variablat e klasës dhe ato të instancës janë të njëjta dhe mund të përdoren pa asnjë ndryshim

d) Nuk ka asnjë ndryshim midis variablave të klasës dhe instancës në Python

Multiple Choice #21 - Zgjidhja

Zgjidhja: a) Variablat e klasës janë të disponueshëm për të gjitha instancat e klasës, ndërsa variablat e instancës janë specifike për çdo objekt

Variablat e klasës janë të përbashkëta për të gjitha instancat, ndërsa variablat e instancës janë specifike për secilin objekt.

Multiple Choice #22

Pyetja. Cili është efekti i përdorimit të metodës super() në Python?

a) I mundëson një klase të thërrasë një metodë të një klase tjetër në hierarkinë e trashëgimisë

b) Ofron një shabllon për përdorimin e metodave të brendshme të klasës

c) Njësoj si thirrja e metodave të instancës për secilën nënklasë

d) Nuk ka ndikim në thirrjen e metodave të klasës

Multiple Choice #22 - Zgjidhja

Zgjidhja: a) I mundëson një klase të thërrasë një metodë të një klase tjetër në hierarkinë e trashëgimisë

Metoda super() mundëson thirrjen e metodave të klasave prind për të përdorur ose mbivendosur funksionalitete në një nënklasë.

Multiple Choice #23

Pyetja. Cili është përkufizimi i një metodë statike në Python?

a) Një metodë që mund të thirret vetëm nga një instancë e objektit
b) Një metodë që mund të thirret direkt nga klasa pa krijuar një objekt
c) Një metodë që nuk ka nevojë për asnjë parameter të jashtëm për të qenë funksionale
d) Një metodë që përkufizohet vetëm për klasa të brendshme

Multiple Choice #23 - Zgjidhja

Zgjidhja: b) Një metodë që mund të thirret direkt nga klasa pa krijuar një objekt

Metodat statike janë të lidhura me klasën dhe mund të thirren direkt nga ajo pa instancuar një objekt.

Multiple Choice #24

Pyetja. Çfarë do të thotë mbivendosja e metodave në Python?

a) Përdorimi i metodave për të bërë mbivendosjen e funksioneve në Python
b) Krijimi i metodave që e mbivendosin një metodë të mëparshme të klasës prind
c) Lidhja e më shumë metodave në një emër të përbashkët
d) Thirrja e metodave të jashtme në të njëjtën kohë

Multiple Choice #24 - Zgjidhja

Zgjidhja: b) Krijimi i metodave që e mbivendosin një metodë të mëparshme të klasës prind

Mbivendosja e metodave ndodh kur një nënklasë ofron një implementim të ri për një metodë që është e pranishme edhe në klasën prind.

Multiple Choice #25

Pyetja. Cili është qëllimi i përdorimit të metodës __str__ në një klasë?

a) Ofron një mënyrë për të kthyer një përfaqësim të lexueshëm të objektit si një string
b) Thërret një metodë tjetër të brendshme për çdo instancë të klasës
c) E shndërron klasën në një formë të integruar për printim
d) Ndihmon në menaxhimin e atributeve të objektit

Multiple Choice #25 - Zgjidhja

Zgjidhja: a) Ofron një mënyrë për të kthyer një përfaqësim të lexueshëm të objektit si një string

Metoda __str__ ofron një mënyrë për të kthyer një përfaqësim të thjeshtë dhe të lexueshëm të një objekti, që mund të përdoret gjatë printimit.

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

True / False #1

Praktika e programimit procedural është e fokusuar në krijimin e objekteve.

True / False #1 - Zgjidhja

Zgjidhja: False. Programimi procedural është i fokusuar në krijimin e funksioneve dhe përdorimin e tyre për të manipuluar të dhënat.

True / False #2

Ripërdorimi i objekteve ka qenë një faktor në rritjen e përdorimit të programimit orientuar nga objektet.

True / False #2 - Zgjidhja

Zgjidhja: True. Ripërdorimi i objekteve është një nga avantazhet kryesore të programimit orientuar nga objektet.

True / False #3

Është një praktikë e zakonshme në programimin orientuar nga objektet që të gjithë atributet e të dhënave të një klase të jenë të aksesueshme nga kodet jashtë klasës.

True / False #3 - Zgjidhja

Zgjidhja: False. Atributet e klasës zakonisht janë të fshehura nga kodi jashtë klasës për të ruajtur inkapsulimin dhe për të parandaluar manipulimet e pakontrolluara.

True / False #4

Një metodë e klasës nuk ka nevojë të ketë parametrin self.

True / False #4 - Zgjidhja

Zgjidhja: False. Parametri `self` është i nevojshëm për të referuar instancën aktuale të klasës dhe për të manipuluar atributet e saj.

True / False #5

Duke filluar emrin e një atributi me dy nënvizime do ta fshehë atë atribut nga kodi jashtë klasës.

True / False #5 - Zgjidhja

Zgjidhja: True. Përdorimi i dy nënvizimeve (p.sh. __atribut) është një teknikë për të bërë atributin privat në Python dhe të paaksesueshëm jashtë klasës.

True / False #6

Nuk mund ta thirrni drejtpërdrejt metodën `__str__`.

True / False #6 - Zgjidhja

Zgjidhja: False. Metoda `__str__` mund të thirret drejtpërdrejt përmes funksionit të integruar `str()`, ose manualisht.

True / False #7

Një mënyrë për të gjetur klasat e nevojshme për një program orientuar nga objektet është të identifikoni të gjitha foljet në një përshkrim të domenit të problemit.

True / False #7 - Zgjidhja

Zgjidhja: False. Klasat zakonisht identifikohen duke gjetur emrat (substantivët) në përshkrimin e problemit, jo foljet.

True / False #8

Polimorfizmi ju lejon të shkruani metoda në një nënklasë që kanë të njëjtin emër si metodat në klasën prind.

True / False #8 - Zgjidhja

Zgjidhja: True. Polimorfizmi lejon që metoda e një nënklase të mbivendoset për të ofruar një implementim të ndryshëm nga ai i klasës prind.

True / False #9

Nuk është e mundur të thirret metoda `__init__` e klasës prind nga metodat `__init__` të një nënklase.

True / False #9 - Zgjidhja

Zgjidhja: False. Metoda `__init__` e klasës prind mund të thirret duke përdorur `super()` nga një nënklasë.

True / False #10

Një nënklasë mund të ketë një metodë me të njëjtin emër si një metodë në klasën prind.

True / False #10 - Zgjidhja

Zgjidhja: True. Një nënklasë mund të ketë një metodë me të njëjtin emër si një metodë e klasës prind dhe ta mbivendosë atë për të ndryshuar funksionalitetin.

True / False #11

Vetëm metoda `__init__` mund të mbivendoset.

True / False #11 - Zgjidhja

Zgjidhja: False. Edhe metodat e tjera mund të mbivendosen në një nënklasë, jo vetëm `__init__`.

True / False #12

Nuk mund të përdorni funksionin `isinstance` për të përcaktuar nëse një objekt është një instancë e një nënklase të një klase.

True / False #12 - Zgjidhja

Zgjidhja: False. Funksioni `isinstance` mund të përdoret për të kontrolluar nëse një objekt është një instancë e një klase ose një nënklase të asaj klase.

True / False #13

Një nënklasë mund të mbivendosë vetëm metodën __init__ të klasës prind.

True / False #13 - Zgjidhja

Zgjidhja: False. Një nënklasë mund të mbivendosë çdo metodë të klasës prind, jo vetëm __init__.

True / False #14

Në Python, një metodë statike mund të thirret vetëm pas krijimit të një objekti të klasës.

True / False #14 - Zgjidhja

Zgjidhja: False. Një metodë statike mund të thirret direkt nga klasa, pa pasur nevojë të krijohet një objekt.

True / False #15

Kur një metodë është mbivendosur në një nënklasë, ajo mund të thirret gjithashtu përmes një instancë të klasës prind.

True / False #15 - Zgjidhja

Zgjidhja: False. Pasi të mbivendoset, metoda e nënklasës do të thirret kur të përdoret instanca e nënklasës, jo e klasës prind.

True / False #16

Metoda __str__ përdoret për të kthyer një përfaqësim të brendshëm të një objekti që është i lexueshëm dhe i printueshëm.

True / False #16 - Zgjidhja

Zgjidhja: True. Metoda __str__ përdoret për të kthyer një përfaqësim të lexueshëm dhe të formatizuar të një objekti që mund të printohet në një mënyrë të kuptueshme.

True / False #17

Në programimin orientuar nga objektet, atributet private janë të dukshme dhe të aksesueshme jashtë klasës.

True / False #17 - Zgjidhja

Zgjidhja: False. Atributet private janë të fshehura dhe nuk mund të aksesohen direkt nga jashtë klasës për të ruajtur inkapsulimin.

True / False #18

Metoda __init__ e klasës është gjithmonë e thirrur kur krijohet një objekt, pavarësisht nëse është thirrur apo jo direkt.

True / False #18 - Zgjidhja

Zgjidhja: True. Metoda __init__ është e thirrur gjithmonë gjatë krijimit të një objekti dhe përdoret për të inicializuar atributet e tij.

True / False #19

Në një marrëdhënie trashëgimie, një nënklasë mund të aksesojë vetëm metodat dhe atributet e klasës prind që janë të public.

True / False #19 - Zgjidhja

Zgjidhja: True. Atributet dhe metodat e prindit që janë të deklaruara si public mund të aksesohen nga nënklasat.

True / False #20

Polimorfizmi i mundëson një nënklase të përdorë metodat e prindit pa pasur nevojë për të krijuar një objekt të klasës prind.

True / False #20 - Zgjidhja

Zgjidhja: False. Polimorfizmi i mundëson një nënklase të përdorë metodat e prindit përmes mbivendosjes dhe mund të thirret përmes objekteve të nënklasës, por jo direkt pa instancë.

True / False #21

Në programimin orientuar nga objektet, klasa prind ka gjithmonë të njëjtin funksionalitet si nënklasat.

True / False #21 - Zgjidhja

Zgjidhja: False. Klasa prind ofron funksionalitete të përgjithshme, por nënklasat mund të shtojnë ose ndryshojnë këto funksionalitete për të ofruar veçori të reja.

True / False #22

Në Python, mund të përdorni operatorin `isinstance()` për të kontrolluar nëse një objekt është instancë e një klase të caktuar.

True / False #22 - Zgjidhja

Zgjidhja: True. `isinstance()` **kontrollon nëse një objekt është instancë e një klase të caktuar ose e një nënklase të saj.**

True / False #23

Metodat private në Python mund të thirren direkt nga jashtë klasës nëse përdoren instanca të kësaj klase.

True / False #23 - Zgjidhja

Zgjidhja: False. Metodat private janë të aksesueshme vetëm brenda klasës dhe nuk mund të thirren jashtë saj.

True / False #24

Në Python, një klasë mund të trashëgojë metodat dhe atributet nga disa klasa prind, që do të thotë se Python mbështet trashëgiminë shumëfishe.

True / False #24 - Zgjidhja

Zgjidhja: True. Python mbështet trashëgiminë shumëfishe, që do të thotë se një klasë mund të trashëgojë nga më shumë se një klasë prind.

True / False #25

Polimorfizmi i mundëson që të krijohen objekte të ndryshme që i përgjigjen të njëjtës metodë në mënyra të ndryshme.

True / False #25 - Zgjidhja

Zgjidhja: True. Polimorfizmi lejon thirrjen e të njëjtës metodë për objekte të ndryshme, duke sjellë rezultatë të ndryshëm bazuar në llojin e objektit që e thërret metodën.

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Pyetje të Shkurtra #1

Pyetja. Çfarë është inkapsulimi?

Pyetje të Shkurtra #1 - Zgjidhja

Përgjigja: Inkapsulimi është parimi i programimit që bën që të dhënat dhe metodat që operojnë me ato të jenë të lidhura dhe të fshehura nga përdoruesit jashtë klasës, duke ndihmuar në mbrojtjen e të dhënave nga modifikimi i paautorizuar.

Pyetje të Shkurtra #2

Pyetja. Pse duhet të fshihen atributet e të dhënave të një objekti nga kodi jashtë klasës?

Pyetje të Shkurtra #2 - Zgjidhja

Përgjigja: Atributet e të dhënave duhet të fshihen për të mbrojtur të dhënat nga ndryshimet e paautorizuara dhe për të siguruar që të dhënat të manipulohen vetëm përmes metodave të kontrolluara të klasës.

Pyetje të Shkurtra #3

Pyetja. Cila është dallimi mes një klase dhe një instance të një klase?

Pyetje të Shkurtra #3 - Zgjidhja

Përgjigja: Klasa është një shabllon për krijimin e objekteve, ndërsa instanca e klasës është një objekt konkret që ka të dhëna dhe funksione të vetat që i përkasin asaj klase.

Pyetje të Shkurtra #4

Pyetja. Deklarata e mëposhtme thërret metodën e një objekti. Cili është emri i metodës? Cili është emri i variablës që referon objektin?

```
wallet.get_dollar()
```

Pyetje të Shkurtra #4 - Zgjidhja

Përgjigja: Emri i metodës është `get_dollar()`, dhe emri i variablës që referon objektin është `wallet`.

Pyetje të Shkurtra #5

Pyetja. Kur ekzekutohet metoda `__init__`, çfarë i referohet parametri self?

Pyetje të Shkurtra #5 - Zgjidhja

Përgjigja: Parametri `self` i referohet instancës aktuale të klasës që po krijohet dhe përdoret për të manipuluar atribute dhe metoda të klasës.

Pyetje të Shkurtra #6

Pyetja. Si fshihni një atribut në një klasë në Python nga kodi jashtë klasës?

Pyetje të Shkurtra #6 - Zgjidhja

Përgjigja: Përdorni dy nënvizime (__) para emrit të atributit për ta bërë atë privat dhe të paaksesueshëm nga jashtë klasës.

Pyetje të Shkurtra #7

Pyetja. Si e thirrni metodën `__str__`?

Pyetje të Shkurtra #7 - Zgjidhja

Përgjigja: Metoda `__str__` thirret automatikisht kur një objekt kalon në funksionin `str()` ose mund të thirret manualisht si çdo metodë tjetër, p.sh. `object.__str__()`.

Pyetje të Shkurtra #8

Pyetja. Çfarë trashëgon një nënklasë nga klasa prind?

Pyetje të Shkurtra #8 - Zgjidhja

Përgjigja: Një nënklasë trashëgon atribute dhe metoda nga klasa prind dhe mund ti mbivendosë ato për të ndryshuar funksionalitetin.

Pyetje të Shkurtra #9

Pyetja. Shikoni përcaktimin e klasës më poshtë. Cili është emri i klasës prind? Cili është emri i nënklasës?

```
1 class Tiger(Felis):
```

Pyetje të Shkurtra #9 - Zgjidhja

Përgjigja: Klasa prind është `Felis`, dhe nënklasa është `Tiger`.

Pyetje të Shkurtra #10

Pyetja. Çfarë është një metodë e mbivendosur?

Pyetje të Shkurtra #10 - Zgjidhja

Përgjigja: Një metodë e mbivendosur është një metodë e një nënklase që ka të njëjtin emër si një metodë në klasën prind dhe mund të ndryshojë implementimin e saj në përputhje me kërkesat e nënklasës.

Pyetje të Shkurtra #11

Pyetja. Çfarë është një metodë statike dhe si mund të përdoret në një klasë?

Pyetje të Shkurtra #11 - Zgjidhja

Përgjigja: Një metodë statike është një metodë që nuk ka nevojë për një instancë të klasës për të u thirrur. Ajo mund të thirret drejtpërdrejt nga klasa dhe është përdorur për funksione që nuk lidhen drejtpërdrejt me objektet e klasës. Ajo mund të krijohet duke përdorur dekoratorin `@staticmethod`.

Pyetje të Shkurtra #12

Pyetja. Cilat janë avantazhet e inkapsulimit në programimin orientuar nga objektet?

Pyetje të Shkurtra #12 - Zgjidhja

Përgjigja: Inkapsulimi siguron mbrojtjen e të dhënave duke i fshehur ato nga përdoruesi dhe mundëson manipulimin e të dhënave vetëm përmes metodave të kontrolluara. Kjo ndihmon në ruajtjen e integritetit të të dhënave dhe ul mundësinë e gabimeve të paautorizuara.

Pyetje të Shkurtra #13

Pyetja. Çfarë ndodh kur një metodë është mbivendosur në një nënklasë dhe ajo thirret përmes një objekti të nënklasës?

Pyetje të Shkurtra #13 - Zgjidhja

Përgjigja: Kur një metodë është mbivendosur në një nënklasë, ajo thirret automatikisht nga objekti i nënklasës dhe implementimi i saj do të përdoret, duke injoruar versionin e metodës së klasës prind. Ky është një shembull i polimorfizmit.

Pyetje të Shkurtra #14

Pyetja. Si mund të kontrolloni nëse një objekt është instancë e një klase të caktuar në Python?

Pyetje të Shkurtra #14 - Zgjidhja

Përgjigja: Përdorni funksionin `isinstance()` për të kontrolluar nëse një objekt është instancë e një klase të caktuar ose e një nënklase të saj. P.sh., `isinstance(my_object, MyClass)`.

Pyetje të Shkurtra #15

Pyetja. Çfarë është polimorfizmi dhe si i ndihmon programet orientuar nga objektet?

Pyetje të Shkurtra #15 - Zgjidhja

Përgjigja: Polimorfizmi është aftësia e një objekti për të marrë forma të ndryshme, duke mundësuar thirrjen e metodave me të njëjtin emër nga objekte të klasave të ndryshme. Kjo e bën kodin më fleksibël dhe të ripërdorshëm.

Pyetje të Shkurtra #16

Pyetja. Si mund të identifikoni nënklasat në një përshkrim të problemit në dizajnin orientuar nga objektet?

Pyetje të Shkurtra #16 - Zgjidhja

Përgjigja: Nënklasat zakonisht identifikohen duke analizuar llojet e veçanta të objekteve që janë më të specializuara se një klasë bazë. Ato shpesh përfaqësojnë një version më të ngushtë të një klase të përgjithshme dhe mund të përshtaten për të ofruar veçori të reja.

Pyetje të Shkurtra #17

Pyetja. Çfarë është një metodë aksesor dhe si përdoret ajo në klasat Python?

Pyetje të Shkurtra #17 - Zgjidhja

Përgjigja: Një metodë aksesor është një metodë që përdoret për të marrë vlerën e një atributi të klasës pa e ndryshuar atë. Ajo përdoret për të siguruar akses të kontrolluar për atributet private të një objekti.

Pyetje të Shkurtra #18

Pyetja. Çfarë është një metodë modifikues dhe si përdoret ajo në klasat Python?

Pyetje të Shkurtra #18 - Zgjidhja

Përgjigja: Një metodë modifikues është një metodë që përdoret për të ndryshuar vlerën e një atributi të klasës. Ajo lejon që të dhënat të modifikohen përmes metodave të kontrolluara, duke ndihmuar në ruajtjen e inkapsulimit.

Pyetje të Shkurtra #19

Pyetja. Pse është e rëndësishme që të përdoret trashëgimia në programimin orientuar nga objektet?

Pyetje të Shkurtra #19 - Zgjidhja

Përgjigja: Trashëgimia lejon që funksionalitetet të ripërdoren dhe të zgjerohet, duke shmangur përsëritjen e kodit dhe duke siguruar një strukturë të pastër dhe të mirëorganizuar. Kjo e bën kodin më të thjeshtë për mirëmbajtje dhe zgjerim.

Pyetje të Shkurtra #20

Pyetja. A mund të përdorni operatorin `in` për të verifikuar nëse një objekt është instancë e një klase në Python?

Pyetje të Shkurtra #20 - Zgjidhja

Përgjigja: Jo, operatori `in` përdoret për të kontrolluar përfshirjen e një elementi në një koleksion (si listë, grup ose fjalor). Për të kontrolluar nëse një objekt është instancë e një klase, duhet të përdorni funksionin `isinstance()`.

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Algoritmi #1

Pyetja. Supozojmë që `my_car` është emri i një variabli që referon një objekt, dhe `go` është emri i një metode. Shkruani një deklaratë që përdor variablën `my_car` për të thirrur metodën `go`. (Nuk keni nevojë të kaloni asnjë argument në metodën `go`.)

Algoritmi #1 - Zgjidhja

Përgjigja:

```
my_car.go()
```

Algoritmi #2

Pyetja. Shkruani një përcaktim klase të quajtur `Book`. Klasa `Book` duhet të ketë atribute të të dhënave për titullin e librit, emrin e autorit dhe emrin e botuesit. Klasa gjithashtu duhet të ketë:

- Metodën `__init__` për klasën. Metoda duhet të pranojë një argument për secilin nga atributet e të dhënave.
- Metoda aksesor dhe modifikues për secilin atribut të të dhënave.
- Një metodë `__str__` që kthen një string që tregon gjendjen e objektit.

Algoritmi #2 - Zgjidhja

-

```
class Book:
    def __init__(self, title, author, publisher):
        self.title = title
        self.author = author
        self.publisher = publisher

    def get_title(self):
        return self.title

    def set_title(self, title):
        self.title = title

    def get_author(self):
        return self.author
```

```
    def get_author(self, author):
```

Algoritmi #3 I

Pyetja. Shikoni përshkrimin e mëposhtëm të një domeni problemi:

> *Bankat ofrojnë këto lloje llogarish për klientët e saj: llogari kursimi, llogari kontrolli dhe llogari tregtare. Klientët lejohen të depozitojnë para në një llogari (duke rritur bilancin e saj), të tërheqin para nga një llogari (duke zvogëluar bilancin e saj), dhe të fitojnë interes në llogari. Çdo llogari ka një normë interesi.*

Supozojmë që po shkruani një program që do të llogarisë shumën e interesit të fituar për një llogari bankare.

- Identifikoni klasat e mundshme në këtë domen problem.
- Përshtatni listën për të përfshirë vetëm klasën ose klasat e nevojshme për këtë problem.
- Identifikoni përgjegjësitë e klasës ose klasave.

Algoritmi #3 - Zgjidhja

Përgjigja:

- Klasat e mundshme janë: `Account`, `SavingsAccount`, `CheckingAccount`, `MoneyMarketAccount`.
- Lista e klasave të nevojshme mund të jetë vetëm `Account` si klasë bazë, dhe nënklasa `SavingsAccount`, `CheckingAccount`, dhe `MoneyMarketAccount`.
- Përgjegjësitë:
  - Llogaritja e bilancit të llogarisë.
  - Llogaritja e interesit të fituar.
  - Depozita dhe tërheqja e parave.

Algoritmi #4

Pyetja. Shkruani linjën e parë të përcaktimit të një klase `Poodle`. Klasa duhet të zgjerojë klasën `Dog`.

Algoritmi #4 - Zgjidhja

```
1 class Poodle(Dog):
```

Algoritmi #5 I

Pyetja. Shikoni përcaktimet e mëposhtme të klasave:

```
class Plant:
    def __init__(self, plant_type):
        self._plant_type = plant_type
    def message(self):
        print("I'm a plant.")

class Tree(Plant):
    def __init__(self):
        Plant.__init__(self, 'tree')
    def message(self):
        print("I'm a tree.")
```

Algoritmi #5 II

Për këto përcaktime klasash, çfarë do të shfaqin këto deklarata?

```
p = Plant('sapling')
t = Tree()
p.message()
t.message()
```

Algoritmi #5 - Zgjidhja

Përgjigja:

```
I'm a plant.
I'm a tree.
```

Algoritmi #6

Pyetja. Shikoni përcaktimin e mëposhtëm të klasës:

```
class Beverage:
    def __init__(self, bev_name):
        self._bev_name = bev_name
```

Shkruani kodin për një klasë të quajtur `Cola` që është një nënklasë e klasës `Beverage`. Metoda `__init__` e klasës `Cola` duhet të thërrasë metodën `__init__` të klasës `Beverage`, duke kaluar `'cola'` si një argument.

Algoritmi #6 - Zgjidhja

Përgjigja:

```
class Cola(Beverage):
    def __init__(self):
        Beverage.__init__(self, 'cola')
```

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Ushtrimi #1 - Pet Class I

Pyetja. Shkruani një klasë të quajtur `Pet`, e cila duhet të ketë këto atribute të të dhënave:

- `__name` (për emrin e një kafshe shtëpiake)
- `__animal_type` (për llojin e kafshës që është një kafshë shtëpiake. Vlera të mundshme janë "Dog", "Cat", dhe "Bird")
- `__age` (për moshën e kafshës shtëpiake)

Ushtrimi #1 - Pet Class II

Klasa gjithashtu duhet të ketë këto metoda:

- `set_name` - Kjo metodë caktuar një vlerë për fushën `__name`.
- `set_animal_type` - Kjo metodë caktuar një vlerë për fushën `__animal_type`.
- `set_age` - Kjo metodë caktuar një vlerë për fushën `__age`.
- `get_name` - Kjo metodë kthen vlerën e fushës `__name`.
- `get_animal_type` - Kjo metodë kthen vlerën e fushës `__animal_type`.
- `get_age` - Kjo metodë kthen vlerën e fushës `__age`.

Ushtrimi #1 - Pet Class III

Pas krijimit të klasës, shkruani një program që krijon një objekt të klasës dhe kërkon përdoruesit të hyjë emrin, llojin, dhe moshën e kafshës së tij. Ky informacion duhet të ruhet si atribute të objektit. Përdorni metodat aksesor për të marrë emrin, llojin dhe moshën e kafshës dhe shfaqni këtë informacion në ekran.

Ushtrimi #1 - Zgjidhja I

```
class Pet:
    def __init__(self, name, animal_type, age):
        self.__name = name;
        self.__animal_type = animal_type
        self.__age = age

    def set_name(self, name):
        self.__name = name

    def set_animal_type(self, animal_type):
        self.__animal_type = animal_type

    def set_age(self, age):
        self.__age = age
```

Ushtrimi #1 - Zgjidhja II

```
    def get_name(self):
        return self.__name

    def get_animal_type(self):
        return self.__animal_type

    def get_age(self):
        return self.__age
```

Ushtrimi #1 - Zgjidhja III

```python
import pet

def main():
    pet_name = input('Enter the name of the pet: ')
    pet_type = input('Enter the type of animal: ')
    pet_age = int(input('Enter the age of the pet: '))

    mypet = pet.Pet(pet_name, pet_type, pet_age)

    print ('Here is the data that you entered: ')
    print (f'Pet name: {mypet.get_name()}')
    print (f'Animal type: {mypet.get_animal_type()}')
    print (f'Age of pet: {mypet.get_age()}')

if __name__ == '__main__':
    main()
```

Ushtrimi #2 - Car Class I

Pyetja. Shkruani një klasë të quajtur `Car` që ka këto atribute të të dhënave:

- `__year_model` (për vitin e modelit të makinës)
- `__make` (për markën e makinës)
- `__speed` (për shpejtësinë aktuale të makinës)

Klasa `Car` duhet të ketë një metodë `__init__` që pranon vitin e modelit dhe markën si argumente. Këto vlera duhet të caktohen për atributet `__year_model` dhe `__make`, dhe gjithashtu duhet të caktohet 0 për atributin `__speed`.

Ushtrimi #2 - Car Class II

Klasa duhet të ketë gjithashtu këto metoda:

- `accelerate` - Metoda duhet të shtojë 5 në atributin `__speed` çdo herë që thirret.
- `brake` - Metoda duhet të ndajë 5 nga atributi `__speed` çdo herë që thirret.
- `get_speed` - Metoda duhet të kthejë shpejtësinë aktuale.

Pastaj, krijoni një program që krijon një objekt `Car` dhe thërret metodën `accelerate` pesë herë. Pas çdo thirrjeje të metodës `accelerate`, merrni shpejtësinë aktuale dhe shfaqni atë. Më pas thirroni metodën `brake` pesë herë. Pas çdo thirrjeje të metodës `brake`, merrni shpejtësinë aktuale dhe shfaqni atë.

Ushtrimi #2 - Zgjidhja I

```
class Car:
    def __init__(self, year_model, make):
        self.__year_model = year_model
        self.__make = make
        self.__speed = 0

    def accelerate(self):
        self.__speed += 5

    def brake(self):
        self.__speed -=5

    def get_speed(self):
        return self.__speed
```

Ushtrimi #2 - Zgjidhja II

```
import car

def main():
    my_car = car.Car('2008', 'Honda Accord')
    print('car is accelerating: ')
    for i in range(5):
        my_car.accelerate()
        print (f'Current speed: {my_car.get_speed()}')
    print()

    print ('car is braking: ')
    for i in range(5):
        my_car.brake()
        print (f'Current speed: {my_car.get_speed()}')

if __name__ == '__main__':
    main()
```

Ushtrimi #3 - Personal Information Class

Pyetja. Dizajni një klasë që mban të dhënat personale si emri, adresa, mosha dhe numri i telefonit. Shkruani metodat e përshtatshme aksesor dhe modifikues. Gjithashtu, shkruani një program që krijon tre instance të klasës. Një instancë duhet të mbajë informacionin tuaj, ndërsa dy të tjerat duhet të mbajnë informacionin e miqve ose të anëtarëve të familjes tuaj.

Ushtrimi #3 - Zgjidhja I

```
class Information:
    def __init__(self, name, address, age, phone_number):
        self.__name = name
        self.__address = address
        self.__age = age
        self.__phone_number = phone_number

    def set_name(self, name):
        self.__name = name

    def set_address(self, address):
        self.__address = address

    def set_age(self, age):
        self.__age = age

    def set_phone_number(self, phone_number):
        self.__phone_number = phone_number
```

Ushtrimi #3 - Zgjidhja II

```
    def get_name(self):
        return self.__name

    def get_address(self):
        return self.__address

    def get_age(self):
        return self.__age

    def get_phone_number(self):
        return self.__phone_number
```

Ushtrimi #3 - Zgjidhja III

```
import info

def main():
    my_info = info.Information('John Doe','111 My Street', 22, '555-555-1281')
    mom_info = info.Information('Mom', '222 Any Street', 52, '555-555-1234')
    sister_info = info.Information('Jane Doe', '333 Her Street', 20, '555-555-4444')

    print('My information:')
    display_info(my_info)
    print()
    print("Mom's information:")
    display_info(mom_info)
    print()
    print ("Sis's infomation:")
    display_info(sister_info)
```

Ushtrimi #3 - Zgjidhja IV

```
def display_info(info):
    print(f'Name: {info.get_name()}')
    print(f'Address: {info.get_address()}')
    print(f'Age: {info.get_age()}')
    print(f'Phone Number: {info.get_phone_number()}')

if __name__ == '__main__':
    main()
```

Ushtrimi #4 - Employee Class I

Pyetja. Shkruani një klasë të quajtur `Employee` që mban të dhënat e mëposhtme për një punonjës:

- emri
- numri i ID
- departamenti
- pozita e punonjësit

Ushtrimi #4 - Employee Class II

Pasi të keni shkruar klasën, shkruani një program që krijon tre objekte `Employee` për të mbajtur të dhënat e mëposhtme:

- Susan Meyers, 47899, Accounting, Vice President
- Mark Jones, 39119, IT, Programmer
- Joy Rogers, 81774, Manufacturing, Engineer

Programi duhet të ruajë këto të dhëna në tre objekte dhe të shfaqë të dhënat për çdo punonjës në ekran.

Ushtrimi #4 - Zgjidhja I

```
class Employee:
    def __init__(self, name, id_number, department, title):
        self.__name = name
        self.__id_number = id_number
        self.__department = department
        self.__title = title

    def set_name(self, name):
        self.__name = name

    def set_id_number(self, id_number):
        self.__id_number = id_number

    def set_department(self, department):
        self.__department = department

    def set_title(self, title):
        self.__title = title
```

Ushtrimi #4 - Zgjidhja II

```
    def get_name(self):
        return self.__name

    def get_id_number(self):
        return self.__id_number

    def get_department(self):
        return self.__department

    def get_title(self):
        return self.__title
```

Ushtrimi #4 - Zgjidhja III

```
    def __str__(self):
        result = 'Name: ' + self.get_name() + \
                 '\nID number: ' + self.get_id_number() + \
                 '\nDepartment: ' + self.get_department() + \
                 '\nTitle: ' + self.get_title()
        return result
```

Ushtrimi #4 - Zgjidhja IV

```
import emp

def main():
    emp1 = emp.Employee('Susan Meyers', '47899', 'Accounting', 'Vice President')
    emp2 = emp.Employee('Mark Jones', '39119', 'IT', 'Programmer')
    emp3 = emp.Employee('Joy Rogers', '81774', 'Manufacturing', 'Engineer')

    print('Employee 1:')
    print(emp1)
    print()
    print('Employee 2:')
    print(emp2)
    print()
    print('Employee 3:')
    print(emp3)

if __name__ == '__main__':
    main()
```

Ushtrimi #4 - Zgjidhja V

Ushtrimi #5 - RetailItem Class I

Pyetja. Shkruani një klasë të quajtur `RetailItem` që mban të dhënat për një artikull në një dyqan. Klasë duhet të mbajë këto të dhëna në atributet:

- përshkrimi i artikullit
- njësitë në inventar
- çmimi

Ushtrimi #5 - RetailItem Class II

Pasi të keni shkruar klasën, shkruani një program që krijon tre objekte `RetailItem` dhe ruan të dhënat e mëposhtme në to:

- Jacket, 12, 59.95
- Designer Jeans, 40, 34.95
- Shirt, 20, 24.95

Ushtrimi #5 - Zgjidhja I

```
class RetailItem:
    def __init__(self, description, inventory, price):
        self.__description = description
        self.__inventory = inventory
        self.__price = price

    def set_description(self, description):
        self.__description = description

    def set_inventory(self, inventory):
        self.__inventory = inventory

    def set_price(self, price):
        self.__price = price
```

Ushtrimi #5 - Zgjidhja II

```
    def get_description(self):
        return self.__description

    def get_inventory(self):
        return self.__inventory

    def get_price(self):
        return self.__price
```

Ushtrimi #5 - Zgjidhja III

```
    def __str__(self):
        result = 'Description: ' + self.get_description() + '\n' + \
                 'Units in inventory: ' + str(self.get_inventory()) + \
                 '\nPrice: $' + str(self.get_price())
        return result
```

Ushtrimi #5 - Zgjidhja IV

```
import retail

def main():
    item1 = retail.RetailItem('Jacket', 12, 59.95)
    item2 = retail.RetailItem('Designer Jeans', 40, 34.95)
    item3 = retail.RetailItem('Shirt', 20, 24.95)

    print ('Retail Item 1: ')
    print (item1)
    print()
    print ('Retail Item 2:')
    print (item2)
    print()
    print ('Retail Item 3:')
    print (item3)
if __name__ == '__main__':
    main()
```

Ushtrimi #6 - Patient Charges I

Pyetja. Shkruani një klasë të quajtur `Patient` që ka atributet për të dhënat e mëposhtme:

- emri i parë, emri i mesëm, dhe emri i fundit
- adresa, qyteti, shteti, dhe kodi ZIP
- numri i telefonit
- emri dhe numri i telefonit të kontaktit emergjent

Ushtrimi #6 - Patient Charges II

Klasë gjithashtu duhet të ketë një metodë `__init__` për të inicializuar çdo atribut. Pastaj, shkruani një klasë të quajtur `Procedure` për procedurat mjekësore që janë kryer për një pacient.

Ushtrimi #6 - Zgjidhja I

```
class Patient:
    def __init__(self, first, middle, last, address, city,
                 state, zip_code, phone, em_contact, em_phone):
        self.__first = first
        self.__middle = middle
        self.__last = last
        self.__address = address
        self.__city = city
        self.__state = state
        self.__zip_code = zip_code
        self.__phone = phone
        self.__em_contact = em_contact
        self.__em_phone = em_phone
```

Ushtrimi #6 - Zgjidhja II

```
    def set_first(self, first):
        self.__first = first

    def set_middle(self, middle):
        self.__middle = middle

    def set_last(self, last):
        self.__last = last

    def set_address(self, address):
        self.__address = address

    def set_city(self, city):
        self.__city = city
```

Ushtrimi #6 - Zgjidhja III

```
    def set_state(self, state):
        self.__state = state

    def set_zip_code(self, zip_code):
        self.__zip_code = zip_code

    def set_phone(self, phone):
        self.__phone = phone

    def set_em_contact(self, em_contact):
        self.__em_contact = em_contact

    def set_em_phone(self, em_phone):
        self.__em_phone = em_phone
```

Ushtrimi #6 - Zgjidhja IV

```
    def get_first(self):
        return self.__first

    def get_middle(self):
        return self.__middle

    def get_last(self):
        return self.__last

    def get_address(self):
        return self.__address

    def get_city(self):
        return self.__city
```

Ushtrimi #6 - Zgjidhja V

```
    def get_state(self):
        return self.__state

    def get_zip_code(self):
        return self.__zip_code

    def get_phone(self):
        return self.__phone

    def get_em_contact(self):
        return self.__em_contact

    def get_em_phone(self):
        return self.__em_phone
```

Ushtrimi #6 - Zgjidhja VI

```
    def __str__(self):
        return 'First Name: ' + self.__first + '\nMiddle Name: ' + self.__middle + \
               '\nLast Name: ' + self.__last + '\nAddress: ' + self.__address + \
               '\nCity: ' + self.__city + '\nState: ' + self.__state + \
               '\nZIP: ' + self.__zip_code + '\nPhone: ' + self.__phone + \
               '\nEmergency Contact: ' + self.__em_contact + \
               '\nEmergency Phone: ' + self.__em_phone + '\n'
```

Ushtrimi #6 - Zgjidhja VII

```
class Procedure:
    def __init__(self, name, date, practitioner, charge):
        self.__name = name
        self.__date = date
        self.__practitioner = practitioner
        self.__charge = charge

    def set_name(self, name):
        self.__name = name

    def set_date(self, date):
        self.__date = date

    def set_practitioner(self, practitioner):
        self.__practitioner = practitioner

    def set_charge(self, charge):
        self.__charge = charge
```

Ushtrimi #6 - Zgjidhja VIII

```
    def get_name(self):
        return self.__name

    def get_date(self):
        return self.__date

    def get_practitioner(self):
        return self.__practitioner

    def get_charge(self):
        return self.__charge
```

Ushtrimi #6 - Zgjidhja IX

```
    def __str__(self):
        return 'Procedure: ' + self.__name + '\nDate: ' + self.__date + \
               '\nPractitioner: ' + self.__practitioner + '\nCharge: ' + \
               format(self.__charge, ',.2f') + '\n'
```

Ushtrimi #6 - Zgjidhja X

```
import procedure
import patient

def main():
    procedure_1 = procedure.Procedure('Physical Exam', '8-24-2019', 'Dr. Irvine', 250.0)
    procedure_2 = procedure.Procedure('X-ray', '8-24-2019', 'Dr. Jamison', 500.0)
    procedure_3 = procedure.Procedure('Blood Test', '8-24-2019', 'Dr. Smith', 200.0)

    pat = patient.Patient('James', 'Edward', 'Jones', '123 Main Street',
                          'Billings', 'Montana', '59000', '406-555-1212',
                          'Jenny Jones', '406-555-1213')

    print(pat)
    print(procedure_1)
    print(procedure_2)
    print(procedure_3)
if __name__ == '__main__':
    main()
```

Ushtrimi #6 - Zgjidhja XI

Ushtrimi #7 - Employee Management System

Pyetja. Përdorni klasën `Employee` për të krijuar një program që ruan objekte `Employee` në një fjalor. Programi duhet të mundësojë:

- Kërkoni për një punonjës
- Shtoni një punonjës të ri
- Ndryshoni emrin, departamentin dhe pozitat e punonjësve
- Fshini një punonjës
- Ruani dhe ngarkoni fjalorin nga një skedar

Ushtrimi #7 - Zgjidhja I

```
import emp
import pickle


LOOK_UP = 1
ADD = 2
CHANGE = 3
DELETE = 4
QUIT = 5

FILENAME = 'employees.dat'
```

Ushtrimi #7 - Zgjidhja II

```
def main():
    employees = load_employees()
    choice = 0

    while choice != QUIT:
        choice = get_user_choice()

        if choice== LOOK_UP:
            look_up(employees)
        elif choice == ADD:
            add(employees)
        elif choice == CHANGE:
            change(employees)
        elif choice == DELETE:
            delete(employees)
    save_employees(employees)
```

Ushtrimi #7 - Zgjidhja III

```
def load_employees():
    try:
        input_file = open(FILENAME, 'rb')
        employee_dict = pickle.load(input_file)
        input_file.close()
    except IOError:
        employee_dict = {}

    return employee_dict
```

Ushtrimi #7 - Zgjidhja IV

```
def get_user_choice():

    print()
    print('Menu')
    print('1. Look up an employee')
    print('2. Add a new employee')
    print('3. Change an existing employee')
    print('4. Delete an employee')
    print('5. Quit the program')
    print()

    choice = int(input('Enter your choice: '))

    while choice < LOOK_UP or choice > QUIT:
        choice = int(input('The choice you entered is invalid. Please enter a valid choice: ')
            )

    return choice
```

Ushtrimi #7 - Zgjidhja V

```
1 def look_up(employees):
2     ID = input('Enter an employee ID number: ')
3     print(employees.get(ID, "The specified ID number was not found"))
```

Ushtrimi #7 - Zgjidhja VI

```
def add(employees):
    name = input('Enter employee name: ')
    ID = input('Enter employee ID number: ')
    department = input('Enter employee department: ')
    title = input('Enter employee title: ')

    new_emp = emp.Employee(name, ID, department, title)

    if ID not in employees:
        employees[ID] = new_emp
        print('The new employee has been added.')
    else:
        print('An employee with that ID already exists.')
```

Ushtrimi #7 - Zgjidhja VII

```
def change(employees):
    ID = input('Enter employee ID number: ')

    if ID in employees:
        name = input('Enter the new name: ')
        department = input('Enter the new department: ')
        title = input('Enter the new title: ')

        new_emp = emp.Employee(name, ID, department, title)

        employees[ID] = new_emp
        print('Employee information updated.')
    else:
        print('The specified ID number was not found.')
```

Ushtrimi #7 - Zgjidhja VIII

```
def delete(employees):
    ID = input('Enter employee ID number: ')

    if ID in employees:
        del employees[ID]
        print('Employee information deleted.')
    else:
        print('The specified ID number was not found.')
```

Ushtrimi #7 - Zgjidhja IX

```
def save_employees(employees):
    output_file = open(FILENAME, 'wb')
    pickle.dump(employees, output_file)
    output_file.close()

if __name__ == '__main__':
    main()
```

Ushtrimi #8 - Cash Register

Pyetja. Shkruani një klasë të quajtur `CashRegister` që mund të përdoret me klasën `RetailItem`.

- Ajo duhet të ketë një metodë `purchase_item` që pranon një objekt `RetailItem`.
- Metoda `get_total` duhet të kthejë totalin e çmimit për të gjitha artikujt.
- Metoda `show_items` duhet të shfaqë të dhënat për artikujt.
- Metoda `clear` duhet të pastronte listën e artikujve.

Përdoreni klasën `CashRegister` në një program që lejon përdoruesin të zgjedhë artikuj për blerje dhe më pas të shfaqë faturën.

Ushtrimi #8 - Zgjidhja I

```
import retail
import cashRegister

PANTS = 1
SHIRT = 2
DRESS = 3
SOCKS = 4
SWEATER = 5
```

Ushtrimi #8 - Zgjidhja II

```
def main():

    pants = retail.RetailItem('Pants', 10, 19.99)
    shirt = retail.RetailItem('Shirt', 15, 12.50)
    dress = retail.RetailItem('Dress', 3, 79.00)
    socks = retail.RetailItem('Socks', 50, 1.00)
    sweater = retail.RetailItem('Sweater', 5, 49.99)

    sale_items = {PANTS:pants, SHIRT:shirt, DRESS:dress, SOCKS:socks, SWEATER:sweater}
    register = cashRegister.CashRegister()
    checkout = 'N'
```

Ushtrimi #8 - Zgjidhja III

```
1    while checkout == 'N':
2        user_choice = get_user_choice()
3        item = sale_items[user_choice]
4        if item.get_inventory() == 0:
5            print('The item is out of stock.')
6        else:
7            register.purchase_item(item)
8            new_item = retail.RetailItem(item.get_description(), item.get_inventory()-1, item.
                 get_price())
9            sale_items[user_choice] = new_item
10           checkout = input('Are you ready to check out (Y/N)? ')
```

Ushtrimi #8 - Zgjidhja IV

```
    print()
    print(f'Your purchase total is: {register.get_total():.2f}')
    print()
    register.show_items()
    register.clear()
```

Ushtrimi #8 - Zgjidhja V

```
def get_user_choice():
    print('Menu')
    print('1. Pants')
    print('2. Shirt')
    print('3. Dress')
    print('4. Socks')
    print('5. Sweater')
    print()

    choice = int(input('Enter the menu number of the item you would like to purchase: '))
    print()

    while choice > SWEATER or choice < PANTS:
        choice = int(input('Please enter a valid item number: '))
        print()

    return choice
```

Ushtrimi #8 - Zgjidhja VI

```
if __name__ == '__main__':
    main()
```

Ushtrimi #9 - Trivia Game

Pyetja. Krijoni një lojë trivia për dy lojtarë që ka 10 pyetje dhe katër mundësi për secilën pyetje.

- Përdorni klasën `Question` për të mbajtur të dhënat për pyetjet.
- Programi duhet të mbajë një numër të pikëve për secilin lojtar.
- Pas të gjitha pyetjeve, duhet të shpallet fituesi.

Ushtrimi #9 - Zgjidhja I

```
class Question:
    def __init__(self, question, answer1, answer2, answer3, answer4, solution):
        self.__question = question
        self.__answer1 = answer1
        self.__answer2 = answer2
        self.__answer3 = answer3
        self.__answer4 = answer4
        self.__solution = solution

    def set_question(self, question):
        self.__question = question

    def set_answer1(self, answer1):
        self.__answer1 = answer1

    def set_answer2(self, answer2):
        self.__answer2 = answer2
```

Ushtrimi #9 - Zgjidhja II

```
    def set_answer3(self, answer3):
        self.__answer3 = answer3

    def set_answer4(self, answer4):
        self.__answer4 = answer4

    def set_solution(self, solution):
        self.__solution = solution

    def get_question(self):
        return self.__question

    def get_answer1(self):
        return self.__answer1

    def get_answer2(self):
        return self.__answer2
```

Ushtrimi #9 - Zgjidhja III

```
    def get_answer3(self):
        return self.__answer3

    def get_answer4(self):
        return self.__answer4

    def get_solution(self):
        return self.__solution

    def __str__(self):
        result = self.get_question() + '\n' + \
                '1. ' + self.get_answer1() + '\n' + \
                '2. ' + self.get_answer2() + '\n' + \
                '3. ' + self.get_answer3() + '\n' + \
                '4. ' + self.get_answer4()
        return result
```

Ushtrimi #9 - Zgjidhja IV

```
    def isCorrect(self, answer):
        return answer==self.get_solution
```

Ushtrimi #9 - Zgjidhja V

```
import question

def main():
    first_points = 0
    second_points = 0
    player = ''

    questions = get_questions()
```

Ushtrimi #9 - Zgjidhja VI

```
1
2   for i in range(10):
3       if i % 2 == 0: player = 'first'
4       else: player = 'second'
5       print(f'Question for the {player} player:')
6
7       current = questions[i]
8       print(current)
9       user_answer = int(input('Enter your solution (a number between 1 and 4): '))
10      if current.isCorrect(user_answer):
11          if player == 'first': first_points += 1
12          else: second_points += 1
13          print('That is the correct answer.')
14          print()
15      else: print(f'That is incorrect. The correct answer is {current.get_solution()}')
16          print()
```

Ushtrimi #9 - Zgjidhja VII

```
    print(f'The first player earned {first_points} points.')
    print(f'The second player earned {second_points} points.')
    if first_points == second_points:
        print('It is a tie.')
    elif first_points > second_points:
        print('The first player wins the game.')
    else:
        print('The second player wins the game.')
```

Ushtrimi #9 - Zgjidhja VIII

```
def get_questions():

    questions = []

    question1 = question.Question('How many days are in a lunar year?',
                                  '354', '365', '243', '379', 1)
    questions.append(question1)
    question2 = question.Question('What is the largest planet?',
                                  'Mars', 'Jupiter', 'Earth',
                                  'Pluto', 2)
    questions.append(question2)
    question3 = question.Question('What is the largest kind of whale?',
                                  'Orca whale', 'Humpback whale',
                                  'Beluga whale', 'Blue whale', 4)
    questions.append(question3)
```

Ushtrimi #9 - Zgjidhja IX

```
    question4 = question.Question('Which dinosaur could fly?',
                                  'Triceratops', 'Tyranosaurus Rex',
                                  'Pteranodon', 'Diplodocus', 3)
    questions.append(question4)
    question5 = question.Question('Which of these Winnie the Pooh characters is a donkey?',
                                  'Pooh', 'Eeyore', 'Piglet',
                                  'Kanga', 2)
    questions.append(question5)
    question6 = question.Question('What is the hottest planet?',
                                  'Mars', 'Pluto', 'Earth',
                                  'Venus', 4)
    questions.append(question6)
```

Ushtrimi #9 - Zgjidhja X

```
    question7 = question.Question('Which dinosaur had the largest brain compared to body size?
        ',
                                  'Troodon', 'Stegosaurus',
                                  'Ichthyosaurus', 'Gigantoraptor', 1)
    questions.append(question7)
    question8 = question.Question('What is the largest type of penguins?',
                                  'Chinstrap penguins',
                                  'Macaroni penguins',
                                  'Emperor penguins',
                                  'White-flippered penguins', 3)
    questions.append(question8)
    question9 = question.Question("Which children's story character is a monkey?",
                                  'Winnie the Pooh',
                                  'Curious George', 'Horton',
                                  'Goofy', 2)
    questions.append(question9)
```

Ushtrimi #9 - Zgjidhja XI

```
    question10 = question.Question('How long is a year on Mars?',
                                   '550 Earth days',
                                   '498 Earth days',
                                   '126 Earth days',
                                   '687 Earth days', 4)
    questions.append(question10)

    return questions

if __name__ == '__main__':
    main()
```

Ushtrimi #10 - Employee and ProductionWorker Classes

Pyetja. Shkruani një klasë `Employee` që mban të dhënat për emrin dhe numrin e punonjësit. Pastaj krijoni një klasë të quajtur `ProductionWorker` që është nënklasë e klasës `Employee` dhe ka këto të dhëna:

- Shift number (shift 1 për ditën, shift 2 për natën)
- Hourly pay rate (tarifa për orën e punës)

Ushtrimi #10 - Zgjidhja I

```
class Employee2:
    def __init__(self, name, id_number):
        self.__name = name
        self.__id_number = id_number

    def set_name(self, name):
        self.__name = name

    def set_id_number(self, id_number):
        self.__id_number = id_number

    def get_name(self):
        return self.__name

    def get_id_number(self):
        return self.__id_number
```

Ushtrimi #10 - Zgjidhja II

```
class ProductionWorker(Employee2):
    def __init__(self, name, id_number, shift_number, pay_rate):
        Employee2.__init__(self, name, id_number)
        self.__shift_number = shift_number
        self.__pay_rate = pay_rate

    def set_shift_number(self, shift_number):
        self.__shift_number = shift_number

    def set_pay_rate(self, pay_rate):
        self.__pay_rate = pay_rate

    def get_shift_number(self):
        return self.__shift_number

    def get_pay_rate(self):
        return self.__pay_rate
```

Ushtrimi #10 - Zgjidhja III

```
import emp2 as emp

def main():
    worker_name = input('Enter the name: ')
    worker_id = input('Enter the ID number: ')
    worker_shift = int(input('Enter the shift number: '))
    worker_pay = float(input('Enter the hourly pay rate: '))

    worker = emp.ProductionWorker(worker_name, worker_id, worker_shift, worker_pay)

    print ('Production worker information:')
    print (f'Name: {worker.get_name()}')
    print (f'ID number: {worker.get_id_number()}')
    print (f'Shift: {worker.get_shift_number()}')
    print (f'Hourly Pay Rate: ${worker.get_pay_rate():,.2f}')

if __name__ == '__main__':
    main()
```

Ushtrimi #10 - Zgjidhja IV

Ushtrimi #11 - ShiftSupervisor Class

Pyetja. Shkruani një klasë të quajtur `ShiftSupervisor` që është një nënklasë e klasës `Employee` dhe ka këto atribute:

- Annual salary (pagë vjetore)
- Annual production bonus (bonus vjetor për prodhimin)

Programi duhet të përdorë një objekt të klasës `ShiftSupervisor`.

Ushtrimi #11 - Zgjidhja I

```
import emp2 as emp

def main():
    super_name = input('Enter the name: ')
    super_id = input('Enter the ID number: ')
    super_salary = float(input('Enter the annual salary: '))
    super_bonus = float(input('Enter the bonus: '))

    supervisor = emp.ShiftSupervisor(super_name, super_id, super_salary, super_bonus)

    print ('Shift supervisor worker information: ')
    print (f'Name: {supervisor.get_name()}')
    print (f'ID number: {supervisor.get_id_number()}')
    print (f'Annual Salary: ${supervisor.get_salary():,.2f}')
    print (f'Annual Production Bonus: ${supervisor.get_bonus():,.2f}')

if __name__ == '__main__':
    main()
```

Ushtrimi #12 - Person and Customer Classes

Pyetja. Shkruani një klasë të quajtur `Person` me atribute për emrin, adresën dhe numrin e telefonit. Më pas shkruani një klasë të quajtur `Customer` që është nënklasë e klasës `Person` dhe ka atribute për numrin e klientit dhe një atribut bool për të treguar nëse klienti dëshiron të jetë në listën e postës.

Ushtrimi #12 - Zgjidhja I

```
class Person:
    def __init__(self, name, address, phone_number):
        self.__name = name
        self.__address = address
        self.__phone_number = phone_number

    def set_name(self, name):
        self.__name = name

    def set_address(self, address):
        self.__address = address

    def set_phone_number(self, phone_number):
        self.__phone_number = phone_number
```

Ushtrimi #12 - Zgjidhja II

```
    def get_name(self):
        return self.__name

    def get_address(self):
        return self.__address

    def get_phone_number(self):
        return self.__phone_number
```

Ushtrimi #12 - Zgjidhja III

```
class Customer(Person):
    def __init__(self, name, address, phone_number, cust_number, on_list):
        Person.__init__(self, name, address, phone_number)
        self.__cust_number = cust_number
        self.__on_list = on_list

    def set_cust_number(self, cust_number):
        self.__cust_number = cust_number

    def set_on_list(self, on_list):
        self.__on_list = on_list

    def get_cust_number(self):
        return self.__cust_number

    def get_on_list(self):
        return self.__on_list
```

Ushtrimi #12 - Zgjidhja IV

```
import person

def main():
    on_list_flag = False

    name = input('Enter the name: ')
    address = input('Enter the address: ')
    phone_number = input('Enter the phone number: ')
    cust_number = input('Enter the customer number: ')
    on_list = input('Does the customer wish to be on the mailing list?(Yes/No) ')

    if on_list == 'Yes': on_list_flag = True
    else: on_list_flag = False
```

Ushtrimi #12 - Zgjidhja V

```
    customer = person.Customer(name, address, phone_number, cust_number, on_list_flag)

    print ('Customer information: ')
    print (f'Name: {customer.get_name()}')
    print (f'Address: {customer.get_address()}')
    print (f'Phone number: {customer.get_phone_number()}')
    print (f'Customer Number: {customer.get_cust_number()}')
    print (f'On Mailing List: {customer.get_on_list()}')

if __name__ == '__main__':
    main()
```

Suksese!